AI BASED PERSONAL FINANCE MANAGER

Methodology, Novelty and Key Contributions

Department of Computer Science and Engineering, TKR College of Engineering & Technology

Batch C11 (CSE-C) · Academic year [ACADEMIC YEAR]

Table 1: Project team

S.No.	Student name	Roll number
1	Mekala Kalyan	23K91A05F0
2	Rajanala Sai Hemanth	23K91A05K3
3	Middala Surya Teja Reddy	23K91A05F2
4	Manga Akash	23K91A05E5

Project guide: N Padmavathi

Chapter and section numbers quoted in the text (for example Chapter 8 or Section 6.2) refer to the full project report, 17.Final Document. Figures and tables in this document are numbered from 1.

1. METHODOLOGY

This document describes how FINJARVIS works from end to end and what is new in it. Part 1 gives the pipeline, the data, the algorithms and the reasons they were chosen; Part 2 sets the work against the base paper and separates what is new from what is standard; Part 3 lists the contributions.

1.1 Overall methodology

FINJARVIS treats personal finance as four small problems that share one database: labelling a transaction, watching a budget, forecasting the next month and summarising health. The base paper [1] uses a large model for each (BERT, Q-learning, LSTM, Isolation Forest); this project uses the simplest inspectable model for categorising, forecasting and scoring, a threshold rule for budgets, and omits anomaly detection. The method has seven steps.

1. **Register and authenticate.** A user signs up with a name, e-mail, username and a password that passes five rules; the password is stored as a bcrypt hash and a login returns a signed token valid for seven days.
2. **Record.** Transactions arrive by form or by CSV or Excel import. Each row needs a date, a description, a positive amount and a type; rows that fail are skipped with a warning that names the row.
3. **Categorise.** A TF-IDF and logistic-regression model labels the description; a manual choice that differs from the prediction is stored and used to retrain the model.
4. **Watch.** Monthly budgets per category are compared with the month's expenses; crossing 75, 90 or 100 % writes a notification.
5. **Forecast.** A least-squares line over the monthly expense totals predicts next month's spending and each category's next-month spend; it refuses to answer with fewer than three months of history.
6. **Score.** Six explainable sub-scores are combined with fixed weights into a 0-100 health score with a rating and the weakest area.
7. **Explain and export.** A rule-routed assistant answers questions from the same records, an optional language model handles questions the rules cannot, and PDF, Excel and CSV reports are produced.

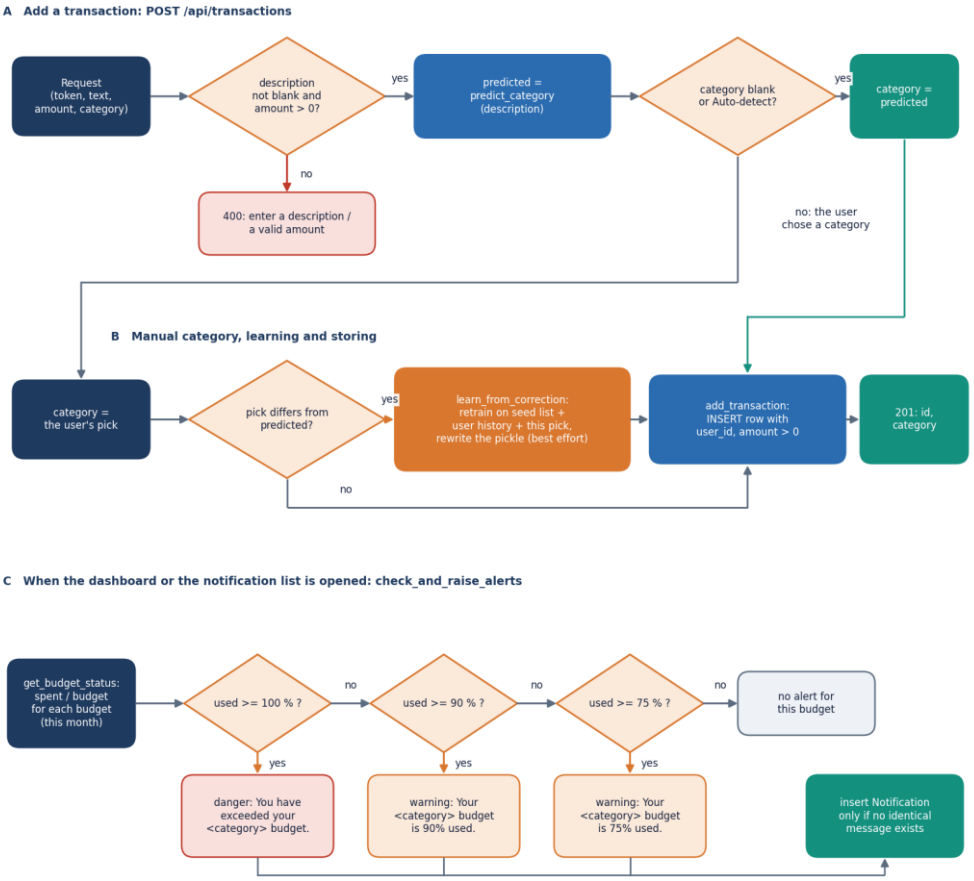


Figure 1: Flowchart of the core algorithms: categoriser, forecast, health score and budget alerts.

1.2 Dataset

FINJARVIS ships **no training corpus beyond a hand-written seed list**: no public dataset, no train-test split inside the application and no licence to cite. Four small data sets are used, all written or generated by the team.

Table 2: Data sets used by the product and by the measurements.

Data set	Size	Used for	Source and caveat
Seed list	51 (description, category) pairs, 11 categories, 3-6 per category	Training the categoriser	Written by the team in `ml/expense_classifier.py`; short English phrases in Indian-merchant style
Sample file	34 rows, Jun-Aug 2026, 10 categories	Demo account and import tests	`data/sample_transactions.csv`; fictional; shifted forward so the newest month is the current month
Demo account	68 transactions, 6 monthly totals (May-Oct 2026), 5 budgets, 3 goals, 4 holdings	Screenshots, forecast backtest, API timing	Sample file plus three synthetic earlier months (seeded generator, about +4 % per month drift); fictional
Probe set	198 descriptions, 11 categories x 18; tiers A, B and C of 66 each	Evaluating the categoriser	`scripts/probe_data.py`, written before scoring and never used for training; **not an independent benchmark**

The seed list is the whole training set of the shipped model; it is balanced only roughly (six examples for Food, Transport and Entertainment, three for Rent, Investments and Salary). The probe set separates three situations: tier A uses wordings close to the seed, tier B uses upper-case statement strings such as `UPI-SWIGGY-8821` that still contain a seed keyword, and tier C uses merchants and phrases the model has never seen. Because the team wrote both seed and probe, tiers A and B are optimistic by construction; tier C is the honest estimate of what a new user would experience.

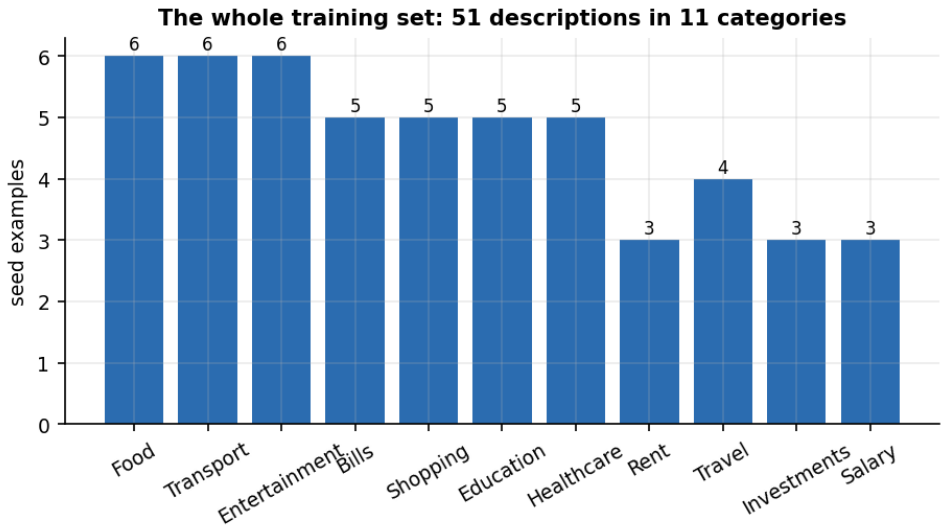


Figure 2: The whole training set of the categoriser: 51 descriptions in 11 categories.

All records in the demo account are fictional. Nothing in the repository, the screenshots or the measurements is a real person's financial data, and no conclusion about real spending behaviour may be drawn from them.

1.3 Preprocessing

Preprocessing is light. For the categoriser the description is stripped of whitespace (an empty one returns 'Other' without calling the model); the `TfidfVectorizer` lowercases it, splits word tokens and builds unigrams and bigrams (`ngram_range=(1, 2)`, `min_df=1`), giving 131 terms on the 51-example seed. No stop-word list, stemming or spelling correction is applied, so `UPI-SWIGGY-8821` becomes the tokens `upi`, `swiggy` and `8821`, and only `swiggy` is known.

In the import pipeline each row is parsed independently: the date must parse with `pandas.to_datetime`, the amount must be a number above zero, the description non-blank, and the type 'income' or 'expense' (anything else is an expense); a blank category is

filled by the categoriser. A failing row is skipped and reported as 'Row n: skipped (invalid date, amount, or missing description)'. Preview and commit parse the same file.

For the forecast the transactions are grouped by calendar month and summed per type; category forecasts pivot the expenses to a month-by-category table with missing months filled with zero. The health score takes the month's income and expenses, the savings rate, the all-time cash balance, the budget statuses, the holdings and the goals; the neutral values for missing data are listed in Section 6.5.3.

1.4 Algorithm choice and justification

Table 3: Algorithms chosen and the alternatives not used.

Task	Chosen	Why	Not used (base paper [1])
Expense categorisation	TF-IDF + multinomial logistic regression [15], [14]	Trains in milliseconds on 51 examples; no GPU; coefficients and probabilities are inspectable; works offline	BERT with 50 categories and SMOTE [8], [13]: needs thousands of labelled rows and a GPU
Spending forecast	Least-squares straight line over monthly totals	With a handful of months a line has two parameters and cannot overfit; the refusal below three months is explicit	LSTM [11]: needs long series; the base paper reports R2 0.63 on a synthetic series
Budget advice	Fixed alert thresholds (75, 90, 100 %) and rule-based suggestions	Transparent; predictable; no reward function to define	Q-learning budget agent [12]
Anomaly detection	Not built	No labelled anomalies and no time to calibrate a contamination rate	Isolation Forest [10]
Health score	Weighted sum of six clipped sub-scores	Every point can be traced to a component; weights sum to one	A learned score would need outcome data that does not exist here

With 51 examples and 131 features a linear classifier is close to the largest model that can be trained without overfitting, and its probabilities could be used for abstention; the product does not, because it always takes the top label (Chapter 8). The straight line for the forecast is likewise a bet on small data: forecasting practice [18], [17] reports that simple methods are hard to beat on short series, and Chapter 8 tests that on this product's own data.

1.5 Mathematical formulation

1.5.1 Expense categoriser

Let a description d be split into terms t (words and word pairs). With n training descriptions and document frequency df(t), scikit-learn's smoothed inverse document frequency and weight are

$$idf(t) = \ln((1 + n) / (1 + df(t))) + 1 \qquad w(t, d) = tf(t, d) \times idf(t) \qquad x_d = w_d / ||w_d||_2 \quad (1)$$

The logistic-regression classifier has one weight vector w_c and intercept b_c per category c in the set of 11 categories. For a description with feature vector x the probability of category c and the predicted label are

$$P(c \mid x) = \exp(w_c \cdot x + b_c) \,/\, \sum_k \exp(w_k \cdot x + b_k) \qquad \text{label}(d) = \operatorname{argmax}_c P(c \mid x) \quad (2)$$

Training minimises the cross-entropy loss with an L2 penalty on the weights (`LogisticRegression(max_iter=1000)`: penalty strength C = 1 and the lbfgs solver, which are the library defaults). If a description contains no known term then x is the zero vector and P(c | x) depends only on the intercepts; in the shipped model the intercept of Transport (0.321) is the largest, so unseen text is labelled Transport. The retraining on a correction adds the pair (description, corrected label) and the user's whole stored history to the 51 seed pairs and refits from scratch.

1.5.2 Spending forecast

Let y_0 ... y_{n-1} be the monthly expense totals in time order, n at least 3. The slope a and intercept b of the least-squares line, the next-month prediction and the trend label are

$$(a, b) = \operatorname{argmin} \sum_t (y_t - (a\,t + b))^2 \qquad \hat{y}_n = \max(0,\, a\,n + b) \qquad \text{trend} = \begin{cases} \text{increasing} & \text{if } a > 1 \\ \text{decreasing} & \text{if } a < -1 \\ \text{stable} & \text{else} \end{cases} \quad (3)$$

The application also reports the mean of the last three months as the 'recent average'. Category forecasts apply the same fit to each category's monthly series. The budget-overrun check flags a category when its predicted spend exceeds its budget for the current month. Forecast quality is reported with the mean absolute error and the mean absolute percentage error [16]:

$$\text{MAE} = \frac{1}{m} \sum |y_i - \hat{y}_i| \qquad \text{MAPE} = \frac{100}{m} \sum \left| \frac{y_i - \hat{y}_i}{y_i} \right| \quad (4)$$

1.5.3 Financial health score

With sub-scores s_k, each clipped to the range 0 to 100, and weights w_k, the score is H = SUM_k w_k s_k, rounded to one decimal. The sub-scores are defined from the savings rate r (in percent), the monthly income I and expenses E, the cash balance Bal, the budgets, the holdings and the goals:

$$s_{\text{save}} = \text{clip}(r \,/\, 20 \times 100) \qquad s_{\text{exp}} = \text{clip}((1 - E \,/\, I) \,/\, 0.5 \times 100) \qquad s_{\text{emerg}} = \text{clip}((\text{Bal} \,/\, E) \,/\, 6 \times 100) \quad (5)$$

$$s_{\text{budget}} = 100 \times \frac{\text{budgets at or under 100 \%}}{\text{all budgets}} \qquad s_{\text{div}} = \text{clip}\left(\frac{\text{distinct asset types}}{5} \times 100\right) \qquad s_{\text{goal}} = \text{clip}(\text{mean percent complete of goals}) \quad (6)$$

Table 4: Weights, neutral values and rating bands of the health score.

Component	Weight	Full marks when	Value when data is missing
Savings rate	0.30	savings rate is 20 % of income or more	0 if income is zero or the rate is negative
Expense ratio	0.20	expenses are half of income or less	0 if income is zero
Budget adherence	0.15	all budgets are at or under 100 %	50 if no budget is set
Emergency fund	0.15	balance covers six months of expenses	50 if there are no expenses
Diversification	0.10	five or more distinct asset types	0 if there are no holdings
Goal progress	0.10	goals are on average complete	50 if no goal is set

The bands are Excellent from 90, Good from 75, Fair from 60, Needs Improvement from 40 and Critical below 40. The lowest component names the 'area to focus on' and supplies the sentence shown beside the score. The budget status itself is computed as percent used = actual spend in the category and month divided by the budget, times 100; the alert thresholds act on that value.

1.5.4 Assistant routing

The assistant is a cascade, not a model. A lowercased question goes to five handlers in order: 'can I save' (the word 'save' and a number), 'reduce expenses', 'health' or 'score', 'where ... spending', and a category name in the question. The first handler that returns text wins. If none does and an `AI_API_KEY` is set, the question and a summary of the user's figures go to a language model; otherwise the fixed help text is returned. Every answer carries the disclaimer.

1.6 Model architecture and training

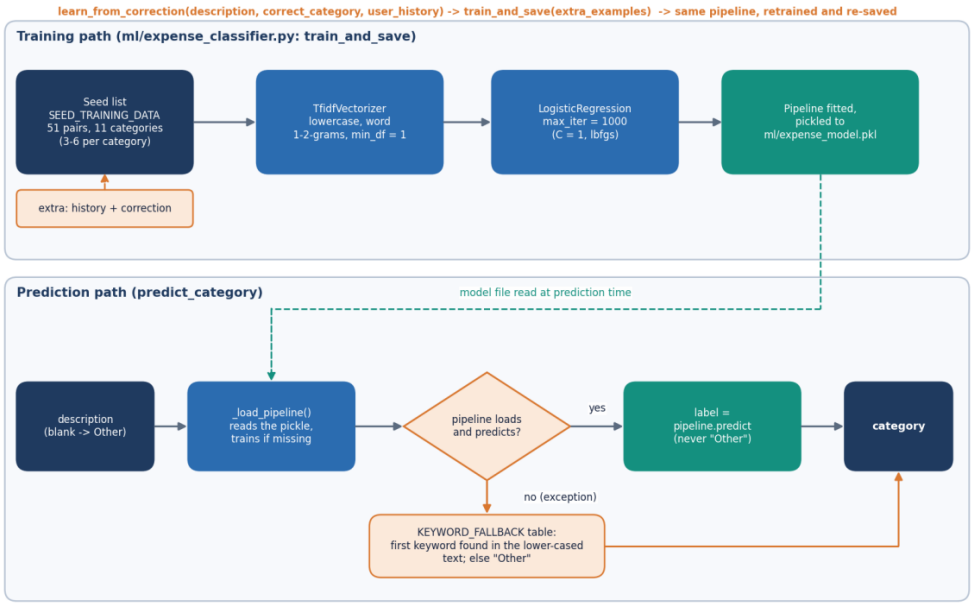


Figure 3: Categoriser pipeline: description, TF-IDF, logistic regression, label, optional correction and retraining.

The shipped categoriser is a scikit-learn `Pipeline` of `tfidf` (`TfidfVectorizer`) and `clf` (`LogisticRegression`), fitted on the 51 seed pairs and pickled to `ml/expense_model.pkl`. Fitting takes 5.2 ms and one prediction 0.84 ms in process (scikit-learn 1.9.1). If the pickle is missing the model is trained on first use; if it cannot be loaded the keyword table (38 keywords) is used, then 'Other'. The repository pickle was written by an older scikit-learn release and loads with an `InconsistentVersionWarning`; retraining removes it. There is no hyper-parameter search or cross-validation, because there is no validation data to drive them.

Table 5: Training configuration of the categoriser.

Item	Value
Vectoriser	`TfidfVectorizer(lowercase=True, ngram_range=(1, 2), min_df=1)`; 131 terms on the seed
Classifier	`LogisticRegression(max_iter=1000)`; default C = 1, lbfgs, multinomial
Training data	51 seed pairs; on a correction: seed + the user's whole history + the corrected pair
Persistence	One pickle file for all users (`ml/expense_model.pkl`, ignored by git)
Fallback	Keyword table, then 'Other'; the model itself never outputs 'Other'
Hardware	Intel Core i5-12500, 15.7 GB RAM, no GPU used; Python 3.11.9

The forecast and the health score have no training stage: each request fits a line with `numpy.polyfit` (the demo account's six months give INR 39,471 against a recent three-month average of INR 36,592), and the weights are constants in `ml/financial_health.py`.

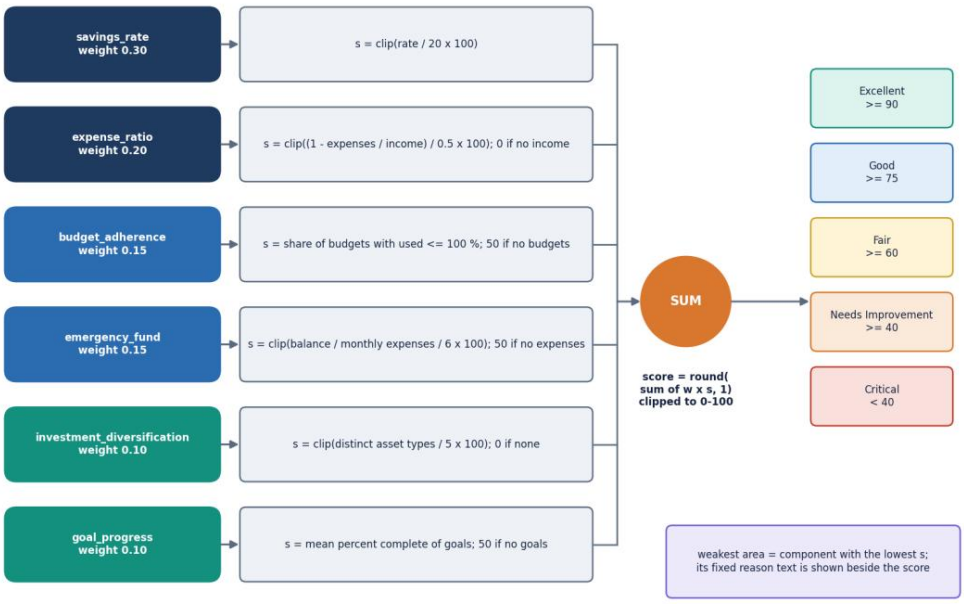


Figure 4: Financial health score model: six weighted components, clipped sum, rating band.

1.7 Evaluation protocol

Because the product has no benchmark data, the evaluation was designed for this project; each design is stated so that the numbers in Chapter 8 can be read correctly.

- **Categoriser.** Accuracy, macro-precision, macro-recall and macro-F1 on the 198-description probe set, reported overall and per tier; compared with the keyword table and with always predicting the most common label (Food). Additional tests: leave-one-out on the 51 seed pairs, accuracy on the 34-row sample file, a confidence-threshold curve, learning from corrections (random subsets of tier C added to the training data, 30 draws), and a repeat-merchant test (one correction, then the same merchant written three other ways).
- **Forecast.** A one-step-ahead rolling-origin backtest on the demo account's six monthly totals (three test points: months 4 to 6), comparing the line with the mean of all history, the mean of the last three months and the last value; plus a synthetic simulation of 2,000 users per setting with 12-month histories (level, linear drift, Gaussian noise and an occasional +50 % spike) to see how the methods behave with more data. The simulation is not real data.
- **Health score.** Sensitivity sweeps from a baseline profile (score 80.7) and 5,000 random profiles to check bounds, rating distribution and monotonicity.
- **Assistant.** 26 questions written by the team (20 answerable from the user's data, 6 off-topic), with two numeric checks against direct sums.
- **Import and API.** A 1,000-row file with 84 injected bad rows; latency of each endpoint on 127.0.0.1 over 30 repetitions (10 for reports and import).
- **Testing.** 38 pytest tests and a 49-check smoke script (Chapter 7).

Two limits apply. The probe set and the seed were written by the same team and the demo account is fictional, so no figure estimates performance on real users; and three test points cannot rank forecasting methods, so the simulation is used to say more. Cross-validation and bootstrap estimators [23] were not applied to the categoriser because 51 examples in 11 classes leave too few per fold; leave-one-out is reported instead and is itself a warning.

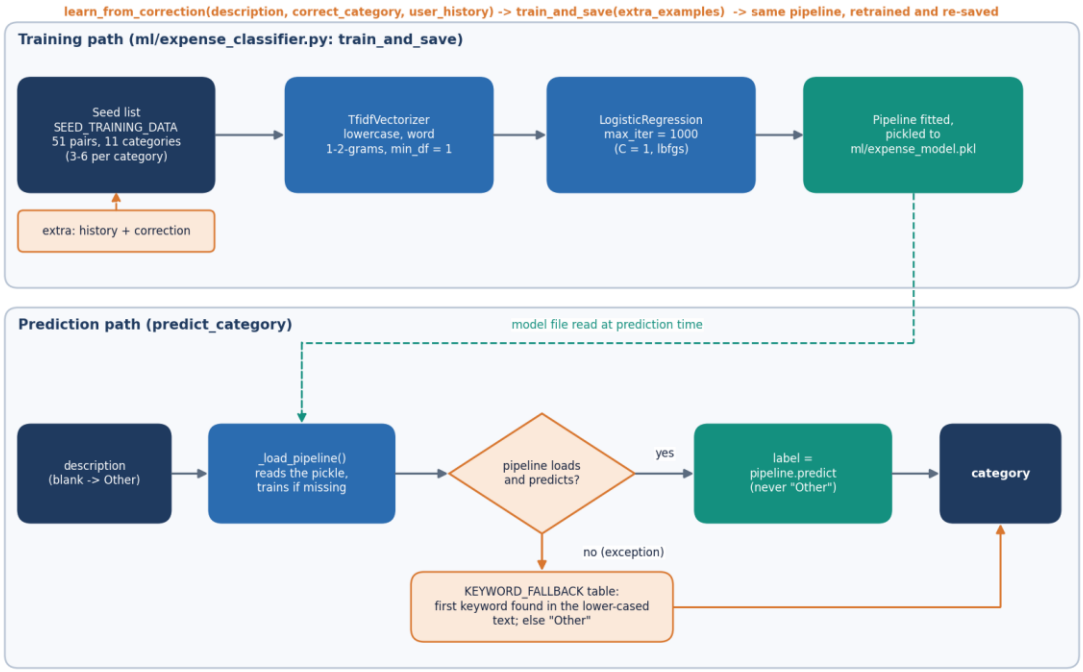


Figure 5: Categoriser pipeline of FINJARVIS: TF-IDF, logistic regression, keyword fallback and learning from corrections.

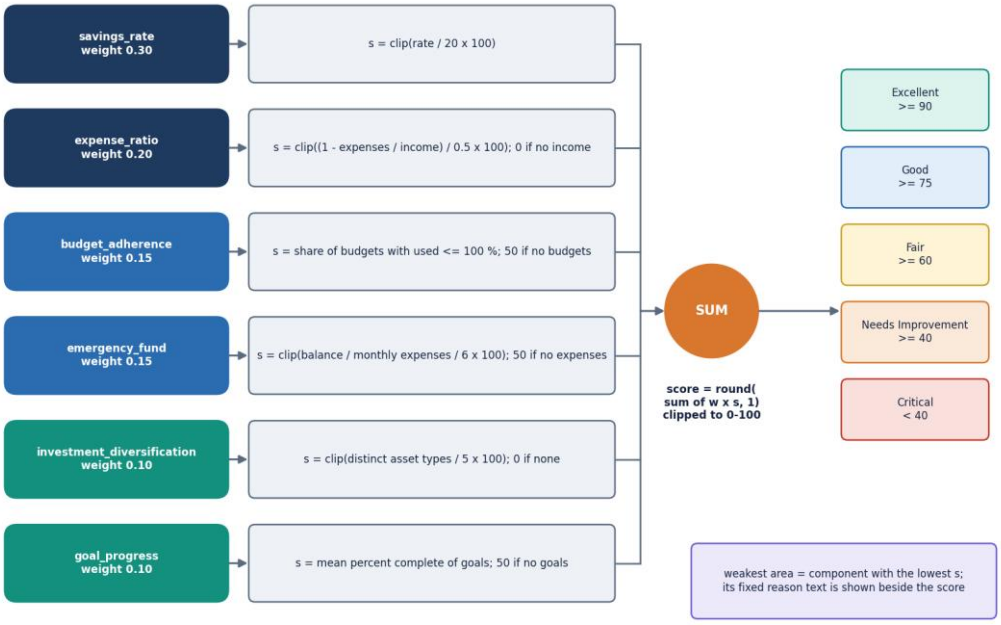


Figure 6: Structure of the six-component financial health score.

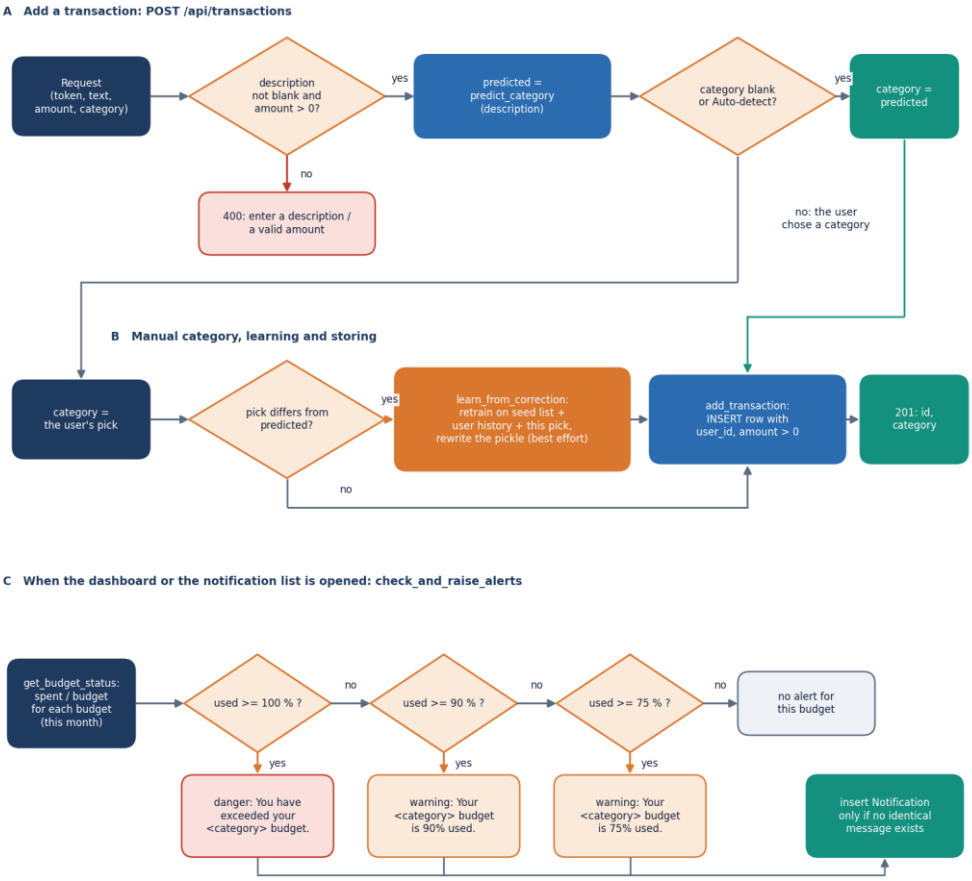


Figure 7: Flowchart of the core algorithm: adding a transaction, auto-categorising it and raising budget alerts.

2. NOVELTY AGAINST THE BASE PAPER

2.1 What the base paper contributes

The registered base paper [1] describes a Smart Personal Finance Assistant built from four separately trained models: a fine-tuned BERT classifier that sorts transactions into 50 expense categories (with SMOTE to balance classes), a Q-learning agent that recommends a budget, an LSTM that forecasts savings, and an Isolation Forest that flags anomalous transactions. It reports an accuracy of 92.3 percent, precision 90.8 percent, recall 91.5 percent and F1 91.1 percent for the classifier, RMSE 137.2, MAE 89.5 and R2 0.63 for the forecaster, and precision 0.93, recall 0.94 and F1 0.94 for the anomaly detector. The data are synthetic or private and are not released, and the paper's interface is a Streamlit prototype.

2.2 What this project does differently

FINJARVIS does not try to match those four models. It is deliberately a smaller system, and it differs from the base paper in the following ways.

- **A complete, runnable web platform.** A person can sign up, log in, import a statement, see a dashboard, set budgets and goals, ask questions and download reports. The base paper describes models and a prototype interface; this project delivers the application around the models.
- **Every module is simple enough to inspect.** The categoriser is TF-IDF with logistic regression, the forecast is a least-squares line, the health score is six weighted sub-scores. Each can be explained in a sentence and reproduced with the code in the repository, at no licence or GPU cost.
- **An explainable financial health score.** Six components with published weights (0.30, 0.20, 0.15, 0.15, 0.10, 0.10) add up to a 0-100 score, and the screen shows the weakest component and a reason. The base paper has no equivalent summary measure.
- **Learning from the user's corrections.** When a manual category differs from the model's prediction, the model is retrained with the user's history and the correction. The measurement shows that this fixes repeat merchants completely and new merchants hardly at all.

- **An assistant grounded in the user's own data.** Questions are routed to five analytics handlers that compute the answer from the user's transactions; an optional language-model layer is used only when a key is configured. Answers carry a disclaimer, and off-topic questions are declined.
- **Measured failure modes.** The categoriser, the forecast, the health score, the assistant and the import pipeline were each measured on inputs the team wrote or simulated, and the places where they fail are reported rather than hidden.

2.3 What is not new, and what is missing

Nothing in the models is new. TF-IDF weighting [15], logistic regression and a linear trend are standard tools, and the evaluation here is not comparable with the base paper's. The categoriser reaches 71.7 percent on a 198-description probe set the team wrote, against 92.3 percent reported for BERT on a different, larger data set with 50 classes. The forecast covers expenses only, and the base paper's savings forecast, reinforcement-learning budget recommender and anomaly detector are not implemented. The batch abstract also names Flask and MySQL; the system was built with FastAPI and SQLite, and the abstract's loan tracking and scheduled digests are not implemented.

Table 6: Base paper modules against this project

Base paper module	Technique in the base paper	In FINJARVIS
Expense categorisation	Fine-tuned BERT, 50 categories, SMOTE	TF-IDF + logistic regression, 11 categories + Other, 51 seed examples, retrains on corrections
Budget recommendation	Q-learning agent	Not implemented: budgets are set by the user; alerts at 75, 90 and 100 percent; rule-based trimming advice
Savings forecasting	LSTM on a synthetic series	Least-squares line over monthly expense totals (expenses, not savings); refuses with fewer than 3 months
Anomaly detection	Isolation Forest, contamination 0.015	Not implemented; only the largest expense in a window is shown
User interface	Streamlit	React + FastAPI web application (the original Streamlit app is kept)
Financial health	Not present	Six-component weighted score with explanation

3. KEY CONTRIBUTIONS

The contributions of the project, stated as what was built and what was measured, are the following.

1. A full-stack personal finance web platform: a React and TypeScript front end with 13 pages, a FastAPI back end with 33 routes, a service layer shared with the original Streamlit application and a six-table SQLite database, with per-user data scoping and bcrypt-hashed passwords.
2. An expense categoriser (TF-IDF word 1-2-grams and logistic regression) with a keyword fallback and retraining on user corrections, evaluated on a 198-description probe set in three difficulty tiers: 100 percent on seed-like text, 97 percent on statement strings and 18.2 percent on unseen merchants.
3. A spending forecast with per-category trends and a budget-overrun check, together with a backtest and a synthetic simulation showing that the linear trend is no better than a plain average until about seven months of history exist.
4. An explainable 0-100 financial health score of six weighted components, checked by sensitivity sweeps and 5,000 random profiles (mean 60.3, range 8.7 to 98.2, no decrease when the savings rate rises).
5. An assistant that answers from the user's own data (14 of 20 answerable test questions handled, all 6 off-topic questions declined), automatic budget alerts, goals, a manual investment portfolio, and PDF, Excel and CSV reports.
6. A CSV and Excel import pipeline with row-level validation: in a 1,000-row test with 84 injected bad rows, all 84 were skipped and 916 were imported and auto-categorised.
7. A test and measurement harness: 38 pytest tests, a 49-check end-to-end smoke test, API latency measurements for every endpoint and a reproducible experiment script that writes `experiments/metrics.json`.
8. An honest account of what the system cannot do: unseen merchants are mislabelled (mostly as Transport), there is no anomaly detection, no reinforcement-learning budgeting and no time-series model beyond a line.